

Databricks End-to-End Project - Restaurant Analytics Platform (Detailed Notes)

Source video: "Databricks End-To-End Project | Zero-To-Expert | Streaming, AI, Lakeflow, Unity Catalog, AI/BI" – <https://www.youtube.com/watch?v=vy4G86q1rQY> (~4 h 17 min) Short version: [claude/restaurant-analytics-e2e-project-notes.md](#)

These notes rebuild the project step by step in my own words. Code is written to be runnable and follows what the video does, but names, endpoints and secrets are placeholders. Check the author's GitHub repo (linked in the video description) for the original files.

Table of contents

0. Big picture: business case, architecture, naming
 1. Synthetic dataset
 2. Azure setup #1: Event Hubs (streaming source)
 3. Azure setup #2: Azure SQL Database (batch + CDC source)
 4. Concept vault: Unity Catalog
 5. Databricks workspace and schemas
 6. Lakeflow Connect: CDC ingestion from SQL Server
 7. Concept vault: Spark Declarative Pipelines (SDP)
 8. Streaming ingestion: Event Hubs → bronze (SDP)
 9. Silver layer: star schema and fact tables
 10. Gold layer: materialized views
 11. Orchestration with Databricks Jobs
 12. AI/BI dashboards
 13. Recap, cost checklist, troubleshooting, interview Q&A
-

0. Big picture

0.1 Business case

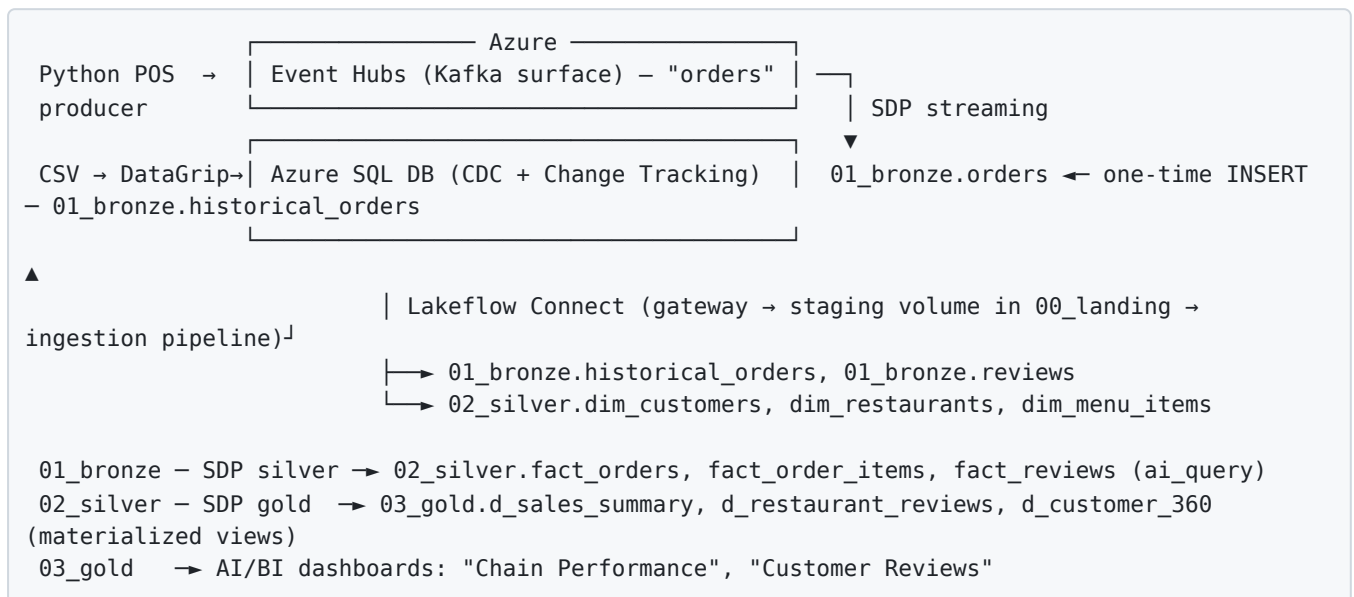
- An Indian restaurant chain with **5 branches in the UAE**: 2 in Abu Dhabi, 2 in Dubai, 1 in Sharjah.
- **500 registered customers, 145 menu items** (spread across restaurants), **~8,000 historical orders** from the last 6 months, and several thousand reviews (only some orders get one).
- Goal: one platform for ingestion (batch + real-time), modeling, AI enrichment and BI, all inside Databricks.

0.2 Why Databricks (the video's pitch)

- One "data intelligence platform" replaces separate tools for data engineering, warehousing, ML and BI.
- Being able to build end-to-end on it is a strong job-market skill.

0.3 Architecture

Layer	Component	Purpose
Source (stream)	Azure Event Hubs, hub <code>orders</code>	Live POS orders every ~3 s
Source (batch)	Azure SQL DB <code>restaurant_ops</code>	customers, restaurants, menu_items, historical_orders, reviews
Ingestion (batch/CDC)	Lakeflow Connect (SQL Server connector)	No-code CDC into bronze and silver
Ingestion (stream)	Spark Declarative Pipeline + Kafka source	Event Hubs → <code>01_bronze.orders</code>
Storage/governance	Unity Catalog + Delta Lake	Schemas <code>00_landing</code> , <code>01_bronze</code> , <code>02_silver</code> , <code>03_gold</code>
Transform	SDP pipelines (silver, gold)	Star schema + aggregates
AI	Mosaic AI Model Serving + <code>ai_query()</code>	Review sentiment + issue tagging in SQL
Orchestration	Databricks Jobs (<code>workflow_daily_master</code>)	ingest → silver → gold
Consumption	Databricks AI/BI dashboards	Chain performance + review insights



0.4 Naming used in the project

Item	Name
Resource group	<code>rg-dbx-project</code>
Event Hubs namespace	<code>eh-namespace-dbx-project</code>

Item	Name
Event hub (topic)	orders
SAS policies	producer-send-policy (Send), databricks-read-policy (Listen)
SQL server / DB	sql-server-ops / restaurant_ops
Databricks workspace	ws-dbx-project (default catalog ws_dbx_project)
UC connection	conn_restaurant_ops
Pipelines	gateway_ingestion (bronze gateway), pipeline_ingestion_bronze, gateway_ingestion_silver, pipeline_ingestion_silver, pipeline_ingestion_eventhub, pipeline_transformation_silver, pipeline_transformation_gold
Job	workflow_daily_master
Cluster policy	minimal-compute-policy

1. Synthetic dataset

1.1 Tables and columns

Table	Key columns
restaurants	restaurant_id, name, city, country, opening_date, phone
menu_items	item_id, restaurant_id, name, category, price
customers	customer_id (e.g. CUST-10000), name, email, city, join_date
historical_orders	order_id, timestamp, restaurant_id, customer_id, order_type (dine_in/takeaway/delivery), items (JSON array string), total_amount, payment_method (cash/card/wallet), order_status
reviews	review_id, order_id, customer_id, restaurant_id, rating (1-5), review_text, review_timestamp

items looks like:

```
[{"item_id":"ITEM-012","name":"Butter Chicken","category":"Main Course","quantity":2,"unit_price":45.0,"subtotal":90.0},
 {"item_id":"ITEM-031","name":"Masala Chai","category":"Beverages","quantity":1,"unit_price":12.0,"subtotal":12.0}]
```

1.2 How the generator works

1. **Restaurants** first (fixed list of 5 with cities).
2. **Menu items** per restaurant (Indian dishes across categories: starters, mains, breads, rice, desserts, beverages).
3. **Customers** (500) with random names, emails, UAE cities, join dates.
4. **Historical orders** read back the generated restaurants/customers/menu items so every foreign key is real (orders only reference existing customers, restaurants and that restaurant's menu).

5. **Reviews** use text templates per star rating; a configurable percentage of orders get a review.
6. Everything is saved as CSV in `data/`. One script (`python synthetic_data.py`) runs them all and overwrites old files.

Tip: you can skip generation and use the CSVs in the repo.

1.3 Minimal generator sketch (my own version)

```
import json, random, uuid, csv
from datetime import datetime, timedelta

ORDER_TYPES = ["dine_in", "takeaway", "delivery"]
PAYMENTS = ["cash", "card", "wallet"]

def make_order(restaurants, customers, menu_by_rest, ts=None):
    r = random.choice(restaurants)
    items = random.sample(menu_by_rest[r["restaurant_id"]], k=random.randint(1, 4))
    lines = []
    for it in items:
        q = random.randint(1, 3)
        lines.append({"item_id": it["item_id"], "name": it["name"], "category": it["category"],
                     "quantity": q, "unit_price": it["price"], "subtotal": round(q *
it["price"], 2)})
    return {
        "order_id": f"ORD-{uuid.uuid4().hex[:10].upper()}",
        "timestamp": (ts or datetime.utcnow()).isoformat(),
        "restaurant_id": r["restaurant_id"],
        "customer_id": random.choice(customers)["customer_id"],
        "order_type": random.choice(ORDER_TYPES),
        "items": json.dumps(lines),
        "total_amount": round(sum(l["subtotal"] for l in lines), 2),
        "payment_method": random.choice(PAYMENTS),
        "order_status": "completed",
    }

# historical: 8,000 orders spread over last 180 days
# hist = [make_order(R, C, M, datetime.utcnow() - timedelta(days=random.uniform(0, 180))) for _
in range(8000)]
```

2. Azure Event Hubs (streaming source)

2.1 Concepts

- **Namespace** = container of event hubs (like a Kafka cluster).
- **Event hub** = a stream (like a Kafka topic). Here: `orders`.
- **Kafka surface**: Event Hubs Standard+ exposes a Kafka-compatible endpoint (port 9093). Any Kafka client, including Spark Structured Streaming's Kafka source, can read/write without you running brokers.

2.2 Portal steps

1. Create **Event Hubs namespace**. - New resource group `rg-dbx-project` (keeps all project resources together; delete the RG at the end to clean up). - Name `eh-namespace-dbx-project`,

region **UAE North**. - Pricing tier **Standard**, because **Basic has no Kafka surface**. - Review + create.

2. Inside the namespace → + **Event Hub** → name `orders`, defaults (1 partition is enough for the demo) → create.
3. Event hub `orders` → **Settings** → **Shared access policies**: - `producer-send-policy` with **Send** only. - `databricks-read-policy` with **Listen** only. - Each policy has a primary **connection string**. Producer shows the Send one; Databricks shows the Listen one. This is least privilege: each client can only do its job.

Producer —(Send conn string)→ Event Hub "orders" ←(Listen conn string)— Databricks

2.3 Local producer (simulated POS)

Setup:

```
conda create -n dbx-project python=3.11 -y
conda activate dbx-project
pip install azure-eventhub python-dotenv
```

`.env` (never commit it):

```
EVENTHUB_CONNECTION_STRING=Endpoint=sb://eh-namespace-dbx-
project.servicebus.windows.net/;SharedAccessKeyName=producer-send-
policy;SharedAccessKey=***;EntityPath=orders
EVENTHUB_NAME=orders
```

`stream_orders.py`:

```
import os, json, time
from dotenv import load_dotenv
from azure.eventhub import EventHubProducerClient, EventData
from generator import load_reference_data, make_order # reuse the same generator

load_dotenv()
producer = EventHubProducerClient.from_connection_string(
    conn_str=os.environ["EVENTHUB_CONNECTION_STRING"],
    eventhub_name=os.environ["EVENTHUB_NAME"],
)
restaurants, customers, menu_by_rest = load_reference_data("data/")
INTERVAL_SECONDS = 3

with producer:
    while True:
        order = make_order(restaurants, customers, menu_by_rest)
        batch = producer.create_batch()
        batch.add(EventData(json.dumps(order))) # 1 event per batch keeps it simple and cheap
        producer.send_batch(batch)
        print("sent", order["order_id"])
        time.sleep(INTERVAL_SECONDS)
```

Key points: - The generator logic is the same as for historical orders, so streamed orders reference real customers, restaurants and items. - Instead of writing a CSV, it sends each order to Event Hubs.

2.4 Validate

Event hub `orders` → **Data Explorer** → "View events" from **newest position**. Match an `order_id` with the producer's console output.

In real life the producer would be the POS system, a mobile app or a web app. Keep the script running only while testing, to save cost.

3. Azure SQL Database (batch + CDC source)

3.1 Create the database

1. **SQL databases → Create:** resource group `rg-dbx-project`, DB name `restaurant_ops`.
2. New server `sql-server-ops`, region UAE North, **SQL authentication** (admin user + password).
3. Workload environment: **Development**.
4. Compute + storage: **General Purpose - Serverless** (billed per vCore-second, auto-pauses). If you stop things promptly the whole project costs roughly 1–2 USD. Basic (~6.46 USD/month) is the fixed alternative.
5. Backup storage redundancy: **Locally-redundant (LRS)**.
6. Networking: - Connectivity: **Public endpoint**. - **Add current client IP address = Yes**, so your laptop/DataGrip can connect through the firewall. - **Allow Azure services and resources to access this server = Yes**, so Databricks (another Azure service) can connect.
7. Review + create.

3.2 Create tables and load CSVs (DataGrip or any SQL client)

1. DataGrip → New data source → **Azure SQL Database** → host `sql-server-ops.database.windows.net`, user, password, database `restaurant_ops` → Test connection.
2. Run the DDL in the `dbo` schema (primary keys matter for Lakeflow Connect):

```
CREATE TABLE dbo.restaurants (  
  restaurant_id VARCHAR(20) PRIMARY KEY, name NVARCHAR(100), city NVARCHAR(50),  
  country NVARCHAR(50), opening_date DATE, phone VARCHAR(30));  
  
CREATE TABLE dbo.menu_items (  
  item_id VARCHAR(20) PRIMARY KEY, restaurant_id VARCHAR(20), name NVARCHAR(100),  
  category NVARCHAR(50), price DECIMAL(10,2));  
  
CREATE TABLE dbo.customers (  
  customer_id VARCHAR(20) PRIMARY KEY, name NVARCHAR(100), email NVARCHAR(150),  
  city NVARCHAR(50), join_date DATE);  
  
CREATE TABLE dbo.historical_orders (  
  order_id VARCHAR(30) PRIMARY KEY, [timestamp] VARCHAR(40), restaurant_id VARCHAR(20),  
  customer_id VARCHAR(20), order_type VARCHAR(20), items NVARCHAR(MAX),  
  total_amount FLOAT, payment_method VARCHAR(20), order_status VARCHAR(20));  
  
CREATE TABLE dbo.reviews (  
  review_id VARCHAR(30) PRIMARY KEY, order_id VARCHAR(30), customer_id VARCHAR(20),  
  restaurant_id VARCHAR(20), rating INT, review_text NVARCHAR(MAX), review_timestamp  
  DATETIME2);
```

3. Right-click each table → **Import data from file** → pick the CSV. Tick "**First row is header**" (otherwise columns show up as C1, C2...) and check the column mapping.
4. Sanity checks:

```
SELECT COUNT(*) FROM dbo.customers;           -- 500
SELECT COUNT(*) FROM dbo.historical_orders;   -- ~8,000
SELECT COUNT(*) FROM dbo.menu_items;         -- 145
SELECT COUNT(*) FROM dbo.restaurants;        -- 5
```

3.3 Enable Change Tracking and CDC (needed by Lakeflow Connect)

1. Run Databricks' **SQL Server utility script** (from the Lakeflow Connect SQL Server docs) against `restaurant_ops`. It installs helper objects, including DDL capture.
2. Enable at database and table level:

```
-- Change tracking (used for tables with a primary key)
ALTER DATABASE restaurant_ops SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 3 DAYS, AUTO_CLEANUP = ON);

ALTER TABLE dbo.customers      ENABLE CHANGE_TRACKING;
ALTER TABLE dbo.restaurants    ENABLE CHANGE_TRACKING;
ALTER TABLE dbo.menu_items     ENABLE CHANGE_TRACKING;
ALTER TABLE dbo.historical_orders ENABLE CHANGE_TRACKING;
ALTER TABLE dbo.reviews        ENABLE CHANGE_TRACKING;

-- CDC (stored procedures)
EXEC sys.sp_cdc_enable_db;
EXEC sys.sp_cdc_enable_table @source_schema = N'dbo', @source_name = N'customers', @role_name = NULL;
-- repeat for each table
```

The video runs the helper statements from the utility script with the DB name, table names, username and password filled in. If CT/CDC is already on, it just reports "already enabled".

At this point both sources are ready: Event Hubs streams live orders, and Azure SQL is ready for batch/CDC ingestion.

4. Concept vault: Unity Catalog

4.1 Four problems UC solves

Problem	What goes wrong without UC	What UC gives you
Data sprawl	Data spread across S3/ADLS buckets, databases and workspaces with no single trusted map. Teams can't find existing data, so they build duplicate ingestion pipelines.	One governed inventory of tables, views, volumes, functions and models across workspaces
Inconsistent access control	Storage, databases and workspaces each have their own permission model. Some users see too much, others bypass controls.	One central role-based access control (GRANT/REVOKE in one place)
Discovery	Analysts and data scientists spend a lot of time just finding and understanding data	Search, tags, comments, lineage , quality signals

Problem	What goes wrong without UC	What UC gives you
Compliance / audit	"Who read dataset X in the last 90 days?" means stitching logs together by hand	User-level audit logs in system tables (<code>system.access.audit</code>), plus billing/usage tables to query cost

Goal: **unified governance for data and AI assets.**

4.2 Object model

```

Metastore (one per region, top-level metadata container)
├─ Catalog          e.g. marketing_dev, marketing_prod, supply_chain_prod
│   └─ Schema       (= database; one use case / project / team sandbox)
│       ├── Tables (managed / external)
│       ├── Views
│       ├── Volumes (files: raw, staging, ML artifacts)
│       ├── Functions
│       └─ Models

```

- Every object is addressed as **catalog.schema.object** (three-level namespace).
- **Metastore**
- Creating a workspace attaches it to the region's metastore, or auto-creates one if none exists.
- **One metastore per region.**
- Metadata lives in the Databricks control plane. Default managed storage is Databricks-managed unless you configure your own storage account (ADLS) or S3 bucket.
- **Catalog** is the main unit of isolation. Common pattern: **business unit x environment** (`<bu>_<env>`), e.g. `marketing_dev`, `marketing_staging`, `marketing_prod`.
- **Schema**: one project/use case, e.g. `marketing_prod.campaign_attribution`.

4.3 Useful SQL

```

CREATE CATALOG IF NOT EXISTS marketing_dev;
CREATE SCHEMA IF NOT EXISTS marketing_dev.project1;
GRANT USE CATALOG ON CATALOG marketing_dev TO `analysts`;
GRANT SELECT ON SCHEMA marketing_dev.project1 TO `analysts`;
SELECT * FROM system.access.audit WHERE event_date >= current_date() - 90;

```

5. Databricks workspace and schemas

1. Azure portal → **Azure Databricks** → **Create**. RG `rg-dbx-project`, name `ws-dbx-project`, region UAE North, **Premium** tier (Unity Catalog and Premium features).
2. Launch workspace → **Catalog**. A catalog named after the workspace (`ws_dbx_project`) already exists. Use it.
3. Create four schemas (UI or SQL):

```

USE CATALOG ws_dbx_project;
CREATE SCHEMA IF NOT EXISTS `00_landing`;    -- staging volume for Lakeflow Connect gateway
CREATE SCHEMA IF NOT EXISTS `01_bronze`;     -- raw
CREATE SCHEMA IF NOT EXISTS `02_silver`;     -- cleaned, typed, modeled (facts + dims)
CREATE SCHEMA IF NOT EXISTS `03_gold`;       -- aggregates for consumption

```

Tip: schema names starting with digits are safest wrapped in backticks in SQL.

4. **SQL warehouse cost control**: edit the **Serverless Starter Warehouse** → size **2X-Small**, auto-stop after short inactivity (the video sets 30 min; lower is cheaper).

6. Lakeflow Connect: CDC ingestion from SQL Server

6.1 What it is

- A **managed ingestion** service in Databricks (Lakeflow = Connect + Declarative Pipelines + Jobs).
- It handles the source connection, change tracking and incremental loading. **No custom code** for supported sources (SQL Server, Salesforce, SharePoint, Workday, ...).
- Inserts/updates in SQL Server flow into Delta tables automatically.

6.2 Architecture: gateway + pipeline

	Ingestion gateway	Ingestion pipeline
Role	Extraction engine: pulls initial snapshots + ongoing CDC from the source DB	Applies staged changes to destination Delta tables
Runs	Continuously	Triggered / scheduled
Compute	Classic compute (a cluster in your subscription, so it can reach private networks)	Serverless (you're asked for a serverless usage/budget policy)
Output	Writes to a UC volume = staging area (here in <code>00_landing</code>)	Streaming tables in bronze/silver

SQL Server → Gateway (classic, continuous) → UC volume (00_landing staging) → Ingestion pipeline (serverless) → Delta tables

6.3 Create the connection and the bronze pipeline

1. **Jobs & Pipelines** → **Create** → **Ingestion pipeline** → **SQL Server**.
2. **Create connection**: auth = username/password, name `conn_restaurant_ops`, host `sql-server-ops.database.windows.net`, port 1433, user, password.
3. **Ingestion gateway** config: name `gateway_ingestion`, staging location catalog `ws_dbx_project`, schema `00_landing`.
4. **Ingestion pipeline** config: name `pipeline_ingestion_bronze`.
5. Create → the gateway starts spinning up a cluster.

6.4 Cost trap: the gateway cluster, and the fix with a cluster policy

- **Compute** → **Job compute** shows the gateway cluster can autoscale to **5 workers**.
- The UI doesn't let you attach a cluster policy to a Lakeflow Connect gateway, so do it through the **CLI**.

Step 1: create a cluster policy (Compute → Policies → Create → Custom → JSON):

```
{
  "num_workers": { "type": "fixed", "value": 1 },
```

```

"driver_node_type_id": { "type": "allowlist", "values": ["Standard_E4d_v4"] },
"node_type_id":        { "type": "allowlist", "values": ["Standard_F4s"] }
}

```

Name: `minimal-compute-policy`. Node types must be ones Lakeflow Connect accepts for gateways.

Step 2: install and configure the Databricks CLI

```

databricks -v          # check it's installed
databricks configure    # host = https://adb-xxxx.azuredatabricks.net
                        # token = PAT from Settings → Developer → Access tokens →
                        # Generate

```

Step 3: find the gateway pipeline ID

```
databricks pipelines list-pipelines
```

Step 4: update the gateway definition with the policy

```

databricks pipelines update <GATEWAY_PIPELINE_ID> --json '{
  "name": "gateway_ingestion",
  "catalog": "ws_dbx_project",
  "schema": "00_landing",
  "gateway_definition": {
    "connection_name": "conn_restaurant_ops",
    "gateway_storage_catalog": "ws_dbx_project",
    "gateway_storage_schema": "00_landing",
    "gateway_storage_name": "gateway_ingestion"
  },
  "clusters": [
    { "label": "default", "policy_id": "<POLICY_ID>", "apply_policy_default_values": true }
  ]
}'

```

Step 5: restart the gateway. Stop it, start a fresh run, then check Job compute: the cluster shows `minimal-compute-policy` and 1 worker.

6.5 Finish the bronze ingestion pipeline

1. Back in the pipeline wizard → **Edit pipeline** → select source tables `dbo.historical_orders` and `dbo.reviews` → destination `ws_dbx_project.01_bronze` (keep the names).
2. Save & continue → validation → **no schedule** → Save & run.
3. Verify:

```

SELECT * FROM ws_dbx_project.`01_bronze`.historical_orders LIMIT 10;
SELECT COUNT(*) FROM ws_dbx_project.`01_bronze`.historical_orders;  -- ~8,000
SELECT COUNT(*) FROM ws_dbx_project.`01_bronze`.reviews;           -- non-zero

```

Also check data types in Catalog Explorer (decimal as expected, `review_text` string, etc.).

This bronze pipeline is a **one-time historical load**. You don't rerun it unless you want a full reload.

⚠ Stop both the gateway and the pipeline afterwards. The gateway is continuous and keeps billing.

6.6 Silver ingestion pipeline (dimension tables)

1. New ingestion pipeline, same connection → pipeline `pipeline_ingestion_silver`, gateway `gateway_ingestion_silver`, staging `00_landing`.
2. Repeat §6.4 steps 3–5 for the new gateway's pipeline ID (same policy).
3. When the gateway is initialized, select tables and **rename destinations**:

Source	Destination (02_silver)
<code>dbo.customers</code>	<code>dim_customers</code>
<code>dbo.restaurants</code>	<code>dim_restaurants</code>
<code>dbo.menu_items</code>	<code>dim_menu_items</code>

4. Save, validate, run. Expected: 500 customers, 5 restaurants, 145 menu items.

6.7 CDC demo: prove changes flow automatically

1. Stop the silver pipeline (keep the silver **gateway running**, because it's the part that captures changes).
2. In Databricks, note the current value:

```
SELECT * FROM ws_dbx_project.`02_silver`.dim_customers WHERE customer_id = 'CUST-10000'; --  
city = Dubai
```

3. In SQL Server:

```
UPDATE dbo.customers SET city = 'Abu Dhabi' WHERE customer_id = 'CUST-10000';  
INSERT INTO dbo.menu_items (item_id, restaurant_id, name, category, price)  
VALUES ('ITEM-999', 'REST-001', 'Test Item', 'Main Course', 49.00);  
SELECT COUNT(*) FROM dbo.menu_items; -- 146
```

4. Run `pipeline_ingestion_silver` → the UI shows **2 upserted records** (1 in `dim_customers`, 1 in `dim_menu_items`).
5. Verify in Databricks: the city is now Abu Dhabi, `COUNT(*)` of `dim_menu_items` = 146, and `ITEM-999` exists.

Takeaway: **SQL Server tracks the change → gateway fetches it → pipeline MERGEs into Delta**. No custom merge code.

⚠ Stop all gateways and pipelines when you're done.

7. Concept vault: Spark Declarative Pipelines (SDP)

(SDP is the open-sourced evolution of Delta Live Tables / DLT; in Databricks it's "Lakeflow Declarative Pipelines".)

7.1 Two ways to build pipelines

	Imperative (notebooks + Jobs)	Declarative (SDP)
You write	Transform code and order, scheduling, retries, cluster size, checkpoints, merge logic	Only table definitions: "this table = this query over those tables"
Dependencies	You wire notebook 1 → 2 → 3 in a job	Engine builds the dependency graph (DAG) from your code
Streaming	You manage checkpoints, triggers, watermarks	Checkpoints and incremental processing are managed
CDC/SCD	Hand-written MERGE, effective dates, out-of-order handling	AUTO CDC (<code>create_auto_cdc_flow / apply_changes</code>)
Strength	Full control and flexibility	Automation, fewer lines, simpler operations

7.2 "Declarative" means

You state **what** (inputs → desired output), not **how** (execution order, parallelism, incremental logic).

Imperative (3 notebooks + a job):

```
# nb1: raw → bronze
spark.read.format("json").load("/raw/orders").write.mode("append").saveAsTable("bronze_orders")
# nb2: bronze → silver
spark.table("bronze_orders").filter("order_id IS NOT NULL").write.mode("overwrite").saveAsTable("silver_orders")
# nb3: silver → gold
spark.table("silver_orders").groupBy("order_date").agg(F.sum("total_amount")).write.mode("overwrite").saveAsTable("gold_daily")
# + Databricks Job: nb1 → nb2 → nb3
```

Declarative (one file, no ordering needed):

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

@dp.table
def bronze_orders():
    return spark.readStream.format("cloudFiles").option("cloudFiles.format",
"json").load("/raw/orders")

@dp.table
def silver_orders():
    # function name = table name (unless name= is given)
    return spark.readStream.table("bronze_orders").filter("order_id IS NOT NULL")

@dp.materialized_view
def gold_daily():
    return
    spark.read.table("silver_orders").groupBy("order_date").agg(F.sum("total_amount").alias("revenue"))
```

The engine sees that `silver_orders` reads `bronze_orders`, and so on, and builds `bronze` → `silver` → `gold` itself.

7.3 SCD Type 2 example

Goal: `dim_customers` keeps history. If a customer moves from India to Singapore, you get two rows: the old one with `__START_AT`/`__END_AT` filled in, and the new one with `__END_AT = NULL` (current).

Imperative: read the CDC stream, join the existing dim, detect inserts/updates/deletes, close old rows (end date, `is_current=false`), insert new rows, handle out-of-order events, write a custom MERGE, write the stream out. Lots of code.

Declarative:

```
dp.create_streaming_table("dim_customers")

dp.create_auto_cdc_flow(
    target="dim_customers",
    source="cdc_customers",
    keys=["customer_id"],
    sequence_by="updated_at",      # resolves ordering / out-of-order events
    stored_as_scd_type=2,
)
```

SQL equivalent:

```
CREATE OR REFRESH STREAMING TABLE dim_customers;
CREATE FLOW customers_cdc AS AUTO CDC INTO dim_customers
FROM STREAM(cdc_customers) KEYS (customer_id) SEQUENCE BY updated_at STORED AS SCD TYPE 2;
```

7.4 When to choose SDP (5 criteria)

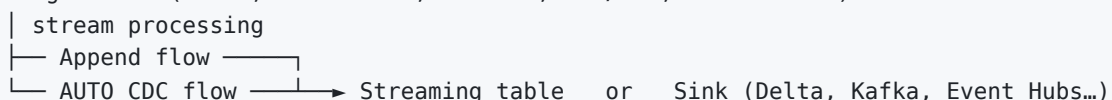
1. **SQL-style ETL/ELT:** table-in → table-out transformations (even if written in PySpark).
2. **Automatic dependency management:** let the platform order tables.
3. **Built-in streaming / incremental processing:** no hand-managed checkpoints or incremental logic.
4. **CDC and SCD patterns:** `AUTO CDC` removes hundreds of lines (SCD1/2, out-of-order events).
5. **Operational simplicity over fine-grained control.**

Prefer notebooks + Jobs when you need low-level control (custom `foreachBatch` logic, unusual sinks, complex non-table side effects).

7.5 Key concepts (Databricks docs diagram)

STREAMING PATH

Streaming source (Kafka, Event Hubs, Kinesis, Pub/Sub, Auto Loader)



BATCH PATH

Batch source (files, tables, streaming table as of a point in time)



- **Flow:** reads a source, applies your logic, writes to a target. Batch or streaming.
- **Append flow:** streaming; appends new rows only, never updates/deletes existing ones.

- **AUTO CDC flow:** streaming; understands CDC events (insert/update/delete + sequence); maintains SCD1/SCD2.
- **Materialized view flow:** batch semantics. The engine tries to process only new/changed source data. A **cost model** chooses incremental vs **full recompute**, so it's not always incremental.
- **Streaming table:** UC-managed Delta table fed by one or more streaming flows. Exactly-once, append-oriented. Best for events, CDC, IoT, ingestion.
- **Materialized view:** UC-managed table whose content is defined by a query. Precomputed, cached, kept up to date as sources change. Best for aggregations and joins/dimensions.
- **Sink:** a generic streaming output outside the pipeline's managed tables.

This project uses **streaming tables** (bronze/silver) and **materialized views** (gold).

8. Streaming ingestion: Event Hubs → 01_bronze.orders

8.1 Setup

1. **Jobs & Pipelines → Create → ETL pipeline** → name `pipeline_ingestion_eventhub`, default catalog `ws_dbx_project`, **default schema** `01_bronze`. (The video first forgot this, and the table landed in `default`.)
2. Start with an empty file → rename it `eventhub.py`.
3. Add an **exploration notebook** `eventhub` to prototype first.
4. Small dev cluster for the notebook: **single node**, no Photon, auto-terminate after 30 min (~0.75 DBU/h).

8.2 Kafka options for Event Hubs

Option	Value	Why
<code>kafka.bootstrap.servers</code>	<code><namespace>.servicebus.windows.net:9093</code>	Event Hubs Kafka endpoint = the namespace
<code>subscribe</code>	<code>orders</code>	Topic = the event hub
<code>kafka.security.protocol</code>	<code>SASL_SSL</code>	TLS + SASL auth
<code>kafka.sasl.mechanism</code>	<code>PLAIN</code>	Event Hubs uses PLAIN with the connection string
<code>kafka.sasl.jaas.config</code>	<code>kafkashaded...PlainLoginModule required username="\$ConnectionString" password="<Listen conn str>;</code>	Uses the databricks-read-policy (Listen)
<code>startingOffsets</code>	<code>earliest</code>	Read from the beginning of the retained stream
<code>maxOffsetsPerTrigger</code>	e.g. <code>1000</code>	Caps records per micro-batch (controls resource use)

Option	Value	Why
failOnDataLoss	true	Fail if expected offsets were deleted/expired instead of silently skipping

8.3 Prototype in the notebook (batch read)

```

from pyspark.sql.functions import *
from pyspark.sql.types import *

EH_NAMESPACE = "eh-namespace-dbx-project"
EH_NAME = "orders"
EH_CONN_STR = dbutils.secrets.get("dbx-project", "eh-listen-conn") # or paste temporarily
while testing

KAFKA_OPTIONS = {
    "kafka.bootstrap.servers": f"{EH_NAMESPACE}.servicebus.windows.net:9093",
    "subscribe": EH_NAME,
    "kafka.security.protocol": "SASL_SSL",
    "kafka.sasl.mechanism": "PLAIN",
    "kafka.sasl.jaas.config": (
        'kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required '
        f'username="{EH_CONN_STR}" password="{EH_CONN_STR}";'
    ),
    "startingOffsets": "earliest",
    "maxOffsetsPerTrigger": "1000",
    "failOnDataLoss": "true",
}

df_raw = spark.read.format("kafka").options(**KAFKA_OPTIONS).load()
display(df_raw) # empty until the producer runs; start stream_orders.py, then re-run

```

The raw Kafka columns are `key`, `value` (binary), `topic`, `partition`, `offset`, `timestamp`, `timestampType`. `key` is null here, which is fine.

Parse:

```

order_schema = StructType([
    StructField("order_id", StringType()),
    StructField("timestamp", StringType()),
    StructField("restaurant_id", StringType()),
    StructField("customer_id", StringType()),
    StructField("order_type", StringType()),
    StructField("items", StringType(), # JSON array kept as string in bronze
    StructField("total_amount", DoubleType()),
    StructField("payment_method", StringType()),
    StructField("order_status", StringType()),
])

df_parsed = (df_raw
    .withColumn("key_str", col("key").cast("string"))
    .withColumn("value_str", col("value").cast("string"))
    .withColumn("data", from_json(col("value_str"), order_schema))
    .select("data.*")

```

```
.withColumnRenamed("timestamp", "order_timestamp"))
display(df_parsed)
```

Flow: binary `value` → string → `from_json` struct → flatten with `data.*` → rename `timestamp` → `order_timestamp`. The bronze orders table has the same columns as `historical_orders`, which makes the later backfill easy.

8.4 Move it into the pipeline

Pipeline settings → Configuration (key/value):

```
eh.namespace      = eh-namespace-dbx-project
eh.name           = orders
eh.connection_string = {{secrets/dbx-project/eh-listen-conn}} # better than plain text
```

eventhub.py:

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import *

EH_NAMESPACE = spark.conf.get("eh.namespace")
EH_NAME      = spark.conf.get("eh.name")
EH_CONN_STR  = spark.conf.get("eh.connection_string")

KAFKA_OPTIONS = { ... } # same dict as in the notebook
order_schema  = StructType([ ... ])

@dp.table(
    name="orders", # lands in the default schema 01_bronze
    table_properties={"quality": "bronze"},
)
def orders():
    df_raw = spark.readStream.format("kafka").options(**KAFKA_OPTIONS).load() # readStream →
    streaming table
    return (df_raw
            .withColumn("value_str", col("value").cast("string"))
            .withColumn("data", from_json(col("value_str"), order_schema))
            .select("data.*")
            .withColumnRenamed("timestamp", "order_timestamp"))
```

Notes: - `@dp.table` + a streaming DataFrame = **streaming table**. Checkpointing is handled for you.
- If `name=` is omitted, the table name = the function name. - Compute: **serverless** by default. -
Always do a **Dry run** first (validates the graph without writing), then **Run pipeline**. First run: e.g.
17 records = everything retained since the producer started, because of `earliest`.

Verify:

```
SELECT COUNT(*) FROM ws_dbx_project.`01_bronze`.orders;
SELECT * FROM ws_dbx_project.`01_bronze`.orders LIMIT 10;
```

Stop the producer and terminate the dev cluster when done.

8.5 One-time backfill of history into the live table

So downstream sees the full picture (6 months of history + live):

```
SELECT COUNT(*) FROM ws_dbx_project.`01_bronze`.orders;    -- e.g. 17

INSERT INTO ws_dbx_project.`01_bronze`.orders
SELECT * FROM ws_dbx_project.`01_bronze`.historical_orders;    -- schemas match column-for-column

SELECT COUNT(*) FROM ws_dbx_project.`01_bronze`.orders;    -- 8,017
```

Caveat: `SELECT *` inserts by **position**, so column order must match. Listing the columns explicitly is safer. Also, a full refresh of this streaming table would drop the backfilled rows. A more robust alternative is a separate **append flow** from `historical_orders` into the same target (`@dp.append_flow(target="orders")` with `once=True`).

Current state: live orders stream in, history is merged in, and Lakeflow Connect syncs the dims.

9. Silver layer: star schema and fact tables

9.1 Data model

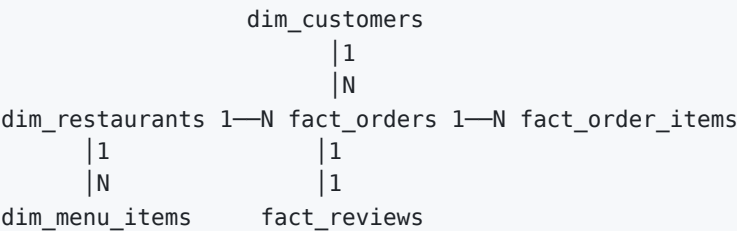


Table	Grain	Source
fact_orders	1 row per order	01_bronze.orders
fact_order_items	1 row per order × item	01_bronze.orders (exploded items)
fact_reviews	1 row per review (+ AI fields)	01_bronze.reviews
dim_customers, dim_restaurants, dim_menu_items	1 row per entity	Lakeflow Connect (\$6.6)

Relationships: - One order → many order items. - One restaurant → many orders. - One customer → many orders. - One order ↔ at most one review. - One restaurant → many menu items.

Pipeline: **ETL pipeline pipeline_transformation_silver**, default schema `02_silver`, one file per table, plus exploration notebooks.

9.2 fact_orders

Target schema:

Column	Type	Logic
order_id	string	
order_timestamp	timestamp	to_timestamp(order_timestamp) (string in bronze)
order_date	date	to_date(order_timestamp)
order_hour	int	hour(order_timestamp)
day_of_week	string	date_format(order_timestamp, 'EEEE') → Monday...
is_weekend	boolean	Saturday or Sunday
restaurant_id, customer_id, order_type	string	
item_count	int	size(from_json(items, item_schema))
total_amount	decimal(10,2)	cast
payment_method, order_status	string	

Notebook prototype:

```

from pyspark.sql.functions import *
from pyspark.sql.types import *

item_schema = ArrayType(StructType([
    StructField("item_id", StringType()),
    StructField("name", StringType()),
    StructField("category", StringType()),
    StructField("quantity", IntegerType()),
    StructField("unit_price", DoubleType()),
    StructField("subtotal", DoubleType()),
]))

df = spark.read.table("ws_dbx_project.`01_bronze`.orders")
df_fact_orders = (df
    .withColumn("order_timestamp", to_timestamp("order_timestamp"))
    .withColumn("order_date", to_date("order_timestamp"))
    .withColumn("order_hour", hour("order_timestamp"))
    .withColumn("day_of_week", date_format("order_timestamp", "EEEE"))
    .withColumn("is_weekend", when(col("day_of_week").isin("Saturday", "Sunday"),
True).otherwise(False))
    .withColumn("items_parsed", from_json("items", item_schema))
    .withColumn("item_count", size("items_parsed"))          # size(), not count()
    .select("order_id", "order_timestamp", "order_date", "order_hour", "day_of_week",
"is_weekend",
        "restaurant_id", "customer_id", "order_type", "item_count",
        col("total_amount").cast("decimal(10,2)").alias("total_amount"),
        "payment_method", "order_status"))
display(df_fact_orders)

```

Mistakes the video hit: `count` instead of `size` for array length; a missing bracket; a wrong alias (`order_date` written where `order_hour` was meant).

Pipeline file `fact_orders.py` with **data quality expectations**:

```

from pyspark import pipelines as dp
from pyspark.sql.functions import *

```

```

from pyspark.sql.types import *

item_schema = ... # as above

@dp.table(name="fact_orders", table_properties={"quality": "silver"})
@dp.expect_all_or_drop({
    "valid_order_id": "order_id IS NOT NULL",
    "valid_order_timestamp": "order_timestamp IS NOT NULL",
    "valid_customer_id": "customer_id IS NOT NULL",
    "valid_restaurant_id": "restaurant_id IS NOT NULL",
    "valid_item_count": "item_count > 0",
    "valid_order_status": "order_status IN
('completed', 'pending', 'preparing', 'ready', 'cancelled')",
    "valid_payment_method": "payment_method IN ('cash', 'card', 'wallet')",
    "valid_total_amount": "total_amount > 0",
})
def fact_orders():
    df = spark.readStream.table("ws_dbx_project.`01_bronze`.orders") # streaming read →
    streaming table
    return (df
        .withColumn("order_timestamp", to_timestamp("order_timestamp"))
        # ... same transformations as the notebook ...
    )

```

(The video writes the streaming read as `dp.read_stream(...)`; `spark.readStream.table(...)` is the current equivalent.)

Result: 8,017 rows, matching bronze. On re-run: **0 new rows** (the streaming checkpoint means already-processed data isn't reprocessed), and **8 expectations registered** in the UI.

9.3 Expectations reference

Decorator (Python)	SQL	Invalid row
<code>@dp.expect(name, cond)</code>	<code>CONSTRAINT x EXPECT (cond)</code>	Kept , counted as a warning
<code>@dp.expect_or_drop</code>	<code>... ON VIOLATION DROP ROW</code>	Dropped
<code>@dp.expect_or_fail</code>	<code>... ON VIOLATION FAIL UPDATE</code>	Pipeline update fails
<code>@dp.expect_all / _all_or_drop / _all_or_fail(dict)</code>	multiple constraints	Same actions, applied to a dict of rules (any failed rule triggers the action)

Metrics (passed/failed counts per rule) show in the pipeline UI and the event log.

9.4 fact_order_items

Purpose: item-level analysis (best sellers, revenue by category).

Column	Type
order_id, item_id, restaurant_id	string
order_timestamp	timestamp
order_date	date

Column	Type
item_name, item_category	string
quantity	int
unit_price, subtotal	decimal(10,2)

fact_order_items.py:

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *
from pyspark.sql.types import *

item_schema = ... # ArrayType(StructType(...)) as above

@dp.table(name="fact_order_items", table_properties={"quality": "silver"})
@dp.expect_all_or_drop({
    "valid_order_id":      "order_id IS NOT NULL",
    "valid_item_id":       "item_id IS NOT NULL",
    "valid_restaurant_id": "restaurant_id IS NOT NULL",
    "valid_order_timestamp": "order_timestamp IS NOT NULL",
    "valid_quantity":      "quantity > 0",
    "valid_unit_price":     "unit_price > 0",
    "valid_subtotal":      "subtotal > 0",
})
def fact_order_items():
    df = spark.readStream.table("ws_dbx_project.`01_bronze`.orders")
    return (df
        .withColumn("order_timestamp", to_timestamp("order_timestamp"))
        .withColumn("order_date", to_date("order_timestamp"))
        .withColumn("items_parsed", from_json("items", item_schema))
        .withColumn("item", explode("items_parsed")) # 1 row per array element
        .select(
            "order_id",
            col("item.item_id").alias("item_id"),
            "restaurant_id", "order_timestamp", "order_date",
            col("item.name").alias("item_name"),
            col("item.category").alias("item_category"),
            col("item.quantity").alias("quantity"),
            col("item.unit_price").cast("decimal(10,2)").alias("unit_price"),
            col("item.subtotal").cast("decimal(10,2)").alias("subtotal"),
        ))
```

- `explode` turns one order row with an N-item array into N rows. The other columns repeat on each row.
- Cast money to **decimal(10,2)** to avoid floating-point rounding issues.
- Result: ~24K rows; 7 expectations registered.

9.5 fact_reviews: AI sentiment with ai_query

Step 1: pick a model. Workspace → **Serving**. It lists Databricks-hosted foundation model endpoints (pay-per-token), e.g. Llama, GPT-OSS, Claude, Gemma. Each has a ready-to-query endpoint. The video picks the cheapest (about 1 DBU per million input tokens).

Step 2: what ai_query does. A built-in SQL function that sends a prompt to a serving endpoint, row by row, and returns the response.

```
SELECT ai_query('databricks-meta-llama-3-3-70b-instruct', 'Describe Databricks in 20 words');
```

Step 3: target columns. Review fields + `analysis_json` + extracted fields:

Column	Meaning
sentiment	positive / neutral / negative
issue_delivery, issue_delivery_reason	late/cold/packaging... and the quoted reason
issue_food_quality, issue_food_quality_reason	taste, freshness, burnt...
issue_pricing, issue_pricing_reason	too expensive / value for money
issue_portion_size, issue_portion_size_reason	small portions

Step 4: prototype in the SQL editor.

```
WITH model_response AS (
  SELECT *,
    ai_query(
      '<endpoint-name>',
      CONCAT(
        'Analyze the following restaurant review and return ONLY a valid JSON object with this structure: ',
        '{"sentiment": "positive|neutral|negative", ',
        '"issue_delivery": true|false, "issue_delivery_reason": "text or null", ',
        '"issue_food_quality": true|false, "issue_food_quality_reason": "text or null", ',
        '"issue_pricing": true|false, "issue_pricing_reason": "text or null", ',
        '"issue_portion_size": true|false, "issue_portion_size_reason": "text or null"}. ',
        'Review: ', review_text)
      ) AS analysis_json
  FROM ws_dbx_project.`01_bronze`.reviews
)
SELECT
  review_id, order_id, customer_id, restaurant_id, rating, review_text, analysis_json,
  get_json_object(analysis_json, '$.sentiment') AS sentiment,
  get_json_object(analysis_json, '$.issue_delivery') AS issue_delivery,
  get_json_object(analysis_json, '$.issue_delivery_reason') AS issue_delivery_reason,
  get_json_object(analysis_json, '$.issue_food_quality') AS issue_food_quality,
  get_json_object(analysis_json, '$.issue_food_quality_reason') AS issue_food_quality_reason,
  get_json_object(analysis_json, '$.issue_pricing') AS issue_pricing,
  get_json_object(analysis_json, '$.issue_pricing_reason') AS issue_pricing_reason,
  get_json_object(analysis_json, '$.issue_portion_size') AS issue_portion_size,
  get_json_object(analysis_json, '$.issue_portion_size_reason') AS issue_portion_size_reason,
  review_timestamp
FROM model_response;
```

- `$` = JSON root; `$.sentiment` = top-level key.
- Keep the raw `analysis_json` for debugging and re-parsing later.
- Insight: a review can be **positive overall** but still flag an issue, e.g. "great food, minor packaging issue" → `issue_delivery = true`. This is useful for operations.
- Improvements: `ai_query(..., responseFormat => ...)` for structured output, or `from_json` with a schema. Also `ai_analyze_sentiment()` exists for sentiment only.

Step 5: SQL file in the silver pipeline, `fact_reviews.sql`:

```
CREATE OR REFRESH STREAMING TABLE fact_reviews (
  CONSTRAINT valid_sentiment EXPECT (sentiment IN ('positive','neutral','negative')) ON
  VIOLATION DROP ROW,
  CONSTRAINT valid_rating EXPECT (rating > 0) ON
  VIOLATION DROP ROW
)
TBLPROPERTIES ('quality' = 'silver')
AS
WITH model_response AS (
  SELECT *, ai_query('<endpoint-name>', CONCAT('...prompt...', review_text)) AS analysis_json
  FROM STREAM(ws_dbx_project.`01_bronze`.reviews) -- STREAM() is required for a streaming
  table
)
SELECT ... FROM model_response; -- same select list as above
```

Gotcha from the video: "cannot create a streaming table append flow from a batch query". A streaming table needs a **streaming source**, so wrap the source in `STREAM(...)`.

Result: e.g. 78 reviews in `fact_reviews`, 2 expectations registered. Because it's a streaming table, each review is sent to the LLM **only once**, which saves cost.

Silver done: typed, validated, modeled, and AI-enriched.

10. Gold layer: materialized views

Pipeline: **ETL pipeline** `pipeline_transformation_gold`, default schema `03_gold`, prototype notebooks per table.

10.1 Streaming table vs materialized view: why gold uses MVs

	Streaming table	Materialized view
Semantics	Append-only, exactly-once processing of new input	Result of a query, always kept correct vs its sources
Read API	<code>spark.readStream...</code> / <code>STREAM()</code>	<code>spark.read...</code> (batch semantics)
Good for	Ingestion, events, CDC, IoT	Aggregations, joins, dimensions
Refresh	Processes only new input rows	Engine refreshes incrementally when the cost model allows , otherwise recomputes fully

The docs say: write MV logic with **batch semantics**, and the engine processes only new data and changes in the sources where it can.

```
from pyspark import pipelines as dp

@dp.materialized_view(name="...", partition_cols=[...], table_properties={"quality": "gold"})
def my_mv():
    return spark.read.table("...") # batch read (video: dp.read)
```

10.2 d_sales_summary (daily chain KPIs)

Column	Logic (from fact_orders, group by order_date)
order_date	group key; partition column
total_orders	countDistinct(order_id)
total_revenue	round(sum(total_amount), 2)
avg_order_value	round(avg(total_amount), 2)
unique_customers	countDistinct(customer_id)
unique_restaurants	countDistinct(restaurant_id)
dine_in_orders	countDistinct(when(order_type=='dine_in', order_id))
takeaway_orders	countDistinct(when(order_type=='takeaway', order_id))
delivery_orders	countDistinct(when(order_type=='delivery', order_id))

d_sales_summary.py:

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *

@dp.materialized_view(
    name="d_sales_summary",
    partition_cols=["order_date"],
    table_properties={"quality": "gold"},
)
def d_sales_summary():
    df = spark.read.table("ws_dbx_project.`02_silver`.fact_orders")
    return (df.groupBy("order_date").agg(
        countDistinct("order_id").alias("total_orders"),
        round(sum("total_amount"), 2).alias("total_revenue"),
        round(avg("total_amount"), 2).alias("avg_order_value"),
        countDistinct("customer_id").alias("unique_customers"),
        countDistinct("restaurant_id").alias("unique_restaurants"),
        countDistinct(when(col("order_type") == "dine_in",
col("order_id"))).alias("dine_in_orders"),
        countDistinct(when(col("order_type") == "takeaway",
col("order_id"))).alias("takeaway_orders"),
        countDistinct(when(col("order_type") == "delivery",
col("order_id"))).alias("delivery_orders"),
    ))
```

Trick: `when(cond, value)` without `otherwise` gives NULL, and `countDistinct` ignores NULLs, so this is a conditional count.

Result: ~182 rows (one per day). The table properties show up in Catalog Explorer.

10.3 d_restaurant_reviews (review KPIs per restaurant)

Column	Logic
restaurant_id, restaurant_name, city	from dim_restaurants
total_reviews	countDistinct(review_id)
avg_rating	round(avg(rating), 2)

Column	Logic
rating_5_count ... rating_1_count	sum(when(rating==n, 1).otherwise(0))
sentiment_positive_count / neutral / negative	sum(when(sentiment=='positive',1).otherwise(0)) ...

d_restaurant_reviews.py:

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *

@dp.materialized_view(name="d_restaurant_reviews", table_properties={"quality": "gold"})
def d_restaurant_reviews():
    reviews = spark.read.table("ws_dbx_project.`02_silver`.fact_reviews")
    stats = reviews.groupBy("restaurant_id").agg(
        countDistinct("review_id").alias("total_reviews"),
        round(avg("rating"), 2).alias("avg_rating"),
        *[sum(when(col("rating") == n, 1).otherwise(0)).alias(f"rating_{n}_count") for n in [5,
4, 3, 2, 1]],
        *[sum(when(col("sentiment") == s, 1).otherwise(0)).alias(f"sentiment_{s}_count")
        for s in ["positive", "neutral", "negative"]],
    )
    restaurants = spark.read.table("ws_dbx_project.`02_silver`.dim_restaurants")
    joined = restaurants.join(stats, "restaurant_id", "left") # restaurants = master list
    count_cols = [c for c in stats.columns if c.endswith("_count")] + ["total_reviews"]
    return joined.select(
        "restaurant_id",
        col("name").alias("restaurant_name"),
        "city",
        *[coalesce(col(c), lit(0)).alias(c) for c in count_cols],
        coalesce(col("avg_rating"), lit(0)).alias("avg_rating"),
    )
```

- **Left join from dim_restaurants** so every restaurant appears even with 0 reviews, and `coalesce(..., 0)` fills the NULLs.
- Bug the video hit: `.alias()` must wrap the whole `sum(...)`, not sit inside `when`.
- Result: 5 rows.
- The table effectively holds the latest snapshot: when a restaurant's reviews change, that row changes. For history, add a `snapshot_date` column and append daily (partitioned by `snapshot_date`), or `MERGE INTO` on `restaurant_id`.

10.4 d_customer_360 (one row per customer)

Used in industry for personalization, loyalty programs and churn analysis.

Group	Columns	Source
Profile	customer_id, customer_name, email, city, join_date	dim_customers
Order stats	total_orders, lifetime_spend, avg_order_value, last_order_date, loyalty_tier	fact_orders
Review stats	total_reviews, avg_rating_given	fact_reviews
Favorites	favorite_restaurant, favorite_item	fact_orders (+ dim_restaurants), fact_order_items ⋈ fact_orders

Group	Columns	Source
Flag	is_vip	lifetime_spend $\geq$ 5,000 (replaces the dropped "at risk: no order in 90 days" idea)

Loyalty tiers (thresholds lowered because there are only ~8K orders):

lifetime_spend	tier
$\geq$ 5,000	Platinum
$\geq$ 2,000	Gold
$\geq$ 1,000	Silver
else	Bronze

d_customer_360.py:

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *
from pyspark.sql.window import Window

S = "ws_dbx_project.`02_silver`"

@dp.materialized_view(name="d_customer_360", table_properties={"quality": "gold"})
def d_customer_360():
    customers = spark.read.table(f"{S}.dim_customers")
    restaurants = spark.read.table(f"{S}.dim_restaurants")
    orders = spark.read.table(f"{S}.fact_orders")
    reviews = spark.read.table(f"{S}.fact_reviews")
    items = spark.read.table(f"{S}.fact_order_items")

    # 1) Order stats
    order_stats = (orders.groupBy("customer_id").agg(
        countDistinct("order_id").alias("total_orders"),
        sum("total_amount").alias("lifetime_spend"),
        round(avg("total_amount"), 2).alias("avg_order_value"),
        max("order_date").alias("last_order_date"))
        .withColumn("loyalty_tier",
            when(col("lifetime_spend") >= 5000, "Platinum")
            .when(col("lifetime_spend") >= 2000, "Gold")
            .when(col("lifetime_spend") >= 1000, "Silver")
            .otherwise("Bronze")))

    # 2) Review stats
    review_stats = reviews.groupBy("customer_id").agg(
        countDistinct("review_id").alias("total_reviews"),
        round(avg("rating"), 2).alias("avg_rating_given"))

    # 3) Favorite restaurant = restaurant with most orders
    w_rest = Window.partitionBy("customer_id").orderBy(col("order_count").desc())
    fav_restaurant = (orders.groupBy("customer_id", "restaurant_id")
        .agg(count("order_id").alias("order_count"))
        .withColumn("rn", row_number().over(w_rest))
        .filter("rn = 1")
        .join(restaurants.select("restaurant_id", col("name").alias("favorite_restaurant")),
            "restaurant_id")
        .select("customer_id", "favorite_restaurant"))

    # 4) Favorite item = item with highest total quantity (needs a fact-to-fact join)
```

```

w_item = Window.partitionBy("customer_id").orderBy(col("item_quantity").desc())
fav_item = (orders.select("order_id", "customer_id")
    .join(items, "order_id")
    .groupBy("customer_id", "item_name")
    .agg(sum("quantity").alias("item_quantity"))
    .withColumn("rn", row_number().over(w_item))
    .filter("rn = 1")
    .select("customer_id", col("item_name").alias("favorite_item")))

# 5) Assemble on the customer spine
c360 = (customers
    .join(order_stats, "customer_id", "left")
    .join(review_stats, "customer_id", "left")
    .join(fav_restaurant, "customer_id", "left")
    .join(fav_item, "customer_id", "left"))

return c360.select(
    "customer_id",
    col("name").alias("customer_name"), "email", "city", "join_date",
    coalesce("total_orders", lit(0)).alias("total_orders"),
    coalesce("lifetime_spend", lit(0)).cast("decimal(10,2)").alias("lifetime_spend"),
    coalesce("avg_order_value", lit(0)).cast("decimal(10,2)").alias("avg_order_value"),
    "last_order_date", # stays NULL if never ordered
    "loyalty_tier",
    coalesce("total_reviews", lit(0)).alias("total_reviews"),
    coalesce("avg_rating_given", lit(0)).cast("decimal(10,2)").alias("avg_rating_given"),
    "favorite_restaurant", "favorite_item",
    when(col("lifetime_spend") >= 5000, True).otherwise(False).alias("is_vip"),
)

```

Design points: - **row_number VS rank/dense_rank**: on ties, rank gives several rows #1. row_number guarantees exactly one favorite. - **Data-modeling lesson**: fact_order_items has no customer_id, so you must join two large fact tables. If the model had denormalized customer_id into order items, that join would disappear. Model for the questions you need to answer. - Customer spine + left joins = every customer appears, including those with no orders. - decimal(10,2) everywhere for money. - Result: 500 rows.

(In the dashboards only d_sales_summary and d_restaurant_reviews are used. d_customer_360 is there to show an industry pattern.)

11. Orchestration: workflow_daily_master

11.1 Pipelines inventory

Pipeline	Role	Run mode
pipeline_ingestion_bronze (+ gateway)	SQL → bronze history + reviews	One-time
pipeline_ingestion_silver (+ gateway)	SQL → silver dims (CDC)	On demand
pipeline_ingestion_eventhub	Event Hubs → 01_bronze.orders	In the job
pipeline_transformation_silver	fact tables	In the job
pipeline_transformation_gold	MVs	In the job

11.2 Build the job

Jobs & Pipelines → **Create** → **Job** → `workflow_daily_master`: 1. Task `pipeline_ingestion_eventhub`: type **Pipeline** → select the Event Hub pipeline. 2. Task `pipeline_transformation_silver`: type Pipeline, **depends on** task 1 (runs only if it succeeded). 3. Task `pipeline_transformation_gold`: type Pipeline, depends on task 2. 4. Optionally add a schedule (daily) and notifications.

```
[ingest eventhub] —success→ [silver transform] —success→ [gold transform]
```

11.3 Test run

1. Start the producer for a short time (e.g. 13 orders), then stop it.
2. **Run now** → View run.
3. Observed: - Ingest: **13** new rows in `01_bronze.orders`. - Silver: `fact_orders` +13, `fact_order_items` +35, `fact_reviews` +0 (no new reviews). - Gold (see *incrementalization* in the pipeline UI):
 - `d_sales_summary` → **INCREMENTAL** (only the affected `order_date` partitions were recomputed).
 - `d_restaurant_reviews` → **NO CHANGE** (no new reviews).
 - `d_customer_360` → **COMPLETE RECOMPUTE**.

11.4 Why did `d_customer_360` fully recompute?

Query the pipeline **event log** for planning decisions:

```
SELECT timestamp, details:planning_information
FROM event_log('<gold_pipeline_id>')      -- or
event_log(TABLE(ws_dbx_project.`03_gold`.d_customer_360))
WHERE event_type = 'planning_information'
ORDER BY timestamp DESC;
```

Put the JSON in a formatter. It says "**incremental plan rejected by the cost model**" → maintenance type **COMPLETE_RECOMPUTE**, with cost estimates. The optimizer builds candidate plans and the cost model chooses. With several joins and window functions, a full recompute was cheaper.

Homework: partition or cluster `d_customer_360` by `customer_id`, stream new orders, and check whether only the affected customers' rows get refreshed (incremental, like `d_sales_summary`).

12. AI/BI dashboards (no Power BI / Tableau)

12.1 Parameters vs field filters

	Parameter	Field filter
Where it acts	Inside the dataset SQL at runtime (:param)	On the result of the dataset after the query ran
Syntax	<code>WHERE order_date BETWEEN :date_range.min AND :date_range.max</code>	Pick a field (e.g. <code>restaurant_name</code>) in a filter widget
Use for	Powerful query-logic control (date ranges, top-N)	Fast interactivity on already-fetched data

	Parameter	Field filter
Used here	Dashboard 1 (date range)	Dashboard 2 (restaurant)

A date-range parameter exposes `.min` and `.max`. Link a **Date range picker** filter widget to the parameter on **every dataset** that should react.

12.2 Dashboard 1: Restaurant Chain Performance

Datasets (each with a date-range parameter):

```
-- ds_sales_summary
SELECT * FROM ws_dbx_project.`03_gold`.d_sales_summary
WHERE order_date BETWEEN :date_range.min AND :date_range.max;

-- ds_fact_orders
SELECT * FROM ws_dbx_project.`02_silver`.fact_orders
WHERE order_date BETWEEN :date_range.min AND :date_range.max;

-- ds_dim_customers
SELECT * FROM ws_dbx_project.`02_silver`.dim_customers
WHERE join_date BETWEEN :date_range.min AND :date_range.max;

-- ds_top10_items
SELECT item_name, SUM(quantity) AS total_quantity_sold
FROM ws_dbx_project.`02_silver`.fact_order_items
WHERE order_date BETWEEN :date_range.min AND :date_range.max
GROUP BY item_name ORDER BY total_quantity_sold DESC LIMIT 10;

-- ds_fact_order_items
SELECT * FROM ws_dbx_project.`02_silver`.fact_order_items
WHERE order_date BETWEEN :date_range.min AND :date_range.max;
```

Widgets | Widget | Type | Dataset | Config | |---|---|---|---| | Total Orders | Counter |
ds_sales_summary | SUM(total_orders) (rows are per day, so sum them) | | Total Revenue | Counter |
ds_sales_summary | SUM(total_revenue) | | Active Customers | Counter | ds_fact_orders | COUNT
DISTINCT(customer_id): customers who ordered | | Avg Order Value | Counter | ds_fact_orders |
AVG(total_amount). **Not** the average of daily averages, which would be wrong | | Unique Customers
| Counter | ds_dim_customers | COUNT DISTINCT(customer_id) = 500 total | | Daily Sales & AOV |
Line | ds_sales_summary | x = order_date (daily), y = total_orders and avg_order_value with
aggregation **None** (already aggregated) | | Top 10 Selling Items | Bar (horizontal) | ds_top10_items |
y = item_name, x = total_quantity_sold (None) | | Order Volume by Day of Week | Bar |
ds_fact_orders | x = day_of_week, y = COUNT DISTINCT(order_id), labels on | | Peak Hour Analysis |
Heatmap | ds_fact_orders | x = order_hour, y = day_of_week, color = COUNT DISTINCT(order_id),
red scheme | | Revenue by Order Type | Bar | ds_fact_orders | y = order_type, x =
SUM(total_amount) | | Revenue by Food Category | Pie | ds_fact_order_items | color =
item_category, angle = SUM(subtotal) | | Date range | Filter: date range picker | all datasets |
parameter `date_range` |

Example insight (synthetic data, so take it lightly): the heatmap shows lunch peaks (~13:00-14:00) on weekdays.

Rename axis titles, then **Publish**. Changing the date range should update every tile.

12.3 Dashboard 2: Customer Reviews Insights

Datasets

```
-- ds_restaurant_reviews (gold, 1 row per restaurant)
SELECT * FROM ws_dbx_project.`03_gold`.d_restaurant_reviews;

-- ds_review_trend
SELECT r.name AS restaurant_name,
       DATE(fr.review_timestamp) AS review_date,
       COUNT(DISTINCT CASE WHEN fr.sentiment = 'positive' THEN fr.order_id END) AS
positive_review_count,
       COUNT(DISTINCT CASE WHEN fr.sentiment = 'neutral' THEN fr.order_id END) AS
neutral_review_count,
       COUNT(DISTINCT CASE WHEN fr.sentiment = 'negative' THEN fr.order_id END) AS
negative_review_count
FROM ws_dbx_project.`02_silver`.fact_reviews fr
JOIN ws_dbx_project.`02_silver`.dim_restaurants r ON fr.restaurant_id = r.restaurant_id
GROUP BY 1, 2
ORDER BY 1, 2;

-- ds_rating_distribution: UNPIVOT with LATERAL VIEW stack
SELECT restaurant_name, rating_label, review_count
FROM ws_dbx_project.`03_gold`.d_restaurant_reviews
LATERAL VIEW stack(5,
  '5 star', rating_5_count,
  '4 star', rating_4_count,
  '3 star', rating_3_count,
  '2 star', rating_2_count,
  '1 star', rating_1_count
) t AS rating_label, review_count;

-- ds_issue_categorization
SELECT r.name AS restaurant_name,
       COUNT(DISTINCT CASE WHEN fr.issue_delivery = 'true' THEN fr.order_id END) AS
count_issue_delivery,
       COUNT(DISTINCT CASE WHEN fr.issue_food_quality = 'true' THEN fr.order_id END) AS
count_issue_food_quality,
       COUNT(DISTINCT CASE WHEN fr.issue_pricing = 'true' THEN fr.order_id END) AS
count_issue_pricing,
       COUNT(DISTINCT CASE WHEN fr.issue_portion_size = 'true' THEN fr.order_id END) AS
count_issue_portion_size
FROM ws_dbx_project.`02_silver`.fact_reviews fr
JOIN ws_dbx_project.`02_silver`.dim_restaurants r ON fr.restaurant_id = r.restaurant_id
GROUP BY 1;

-- ds_recent_positive (duplicate and change to 'negative' for ds_recent_negative)
SELECT r.name AS restaurant_name, fr.review_text, fr.review_timestamp
FROM ws_dbx_project.`02_silver`.fact_reviews fr
JOIN ws_dbx_project.`02_silver`.dim_restaurants r ON fr.restaurant_id = r.restaurant_id
WHERE fr.sentiment = 'positive' AND fr.review_text IS NOT NULL
ORDER BY fr.review_timestamp DESC;
```

- `stack(n, label1, val1, ...)` produces `n` rows per input row. It's the Spark SQL way to **unpivot** (like pandas `melt`). Spark 3.4+ also has an `UNPIVOT` clause.
- The video's issue query first failed because of a wrong column name (`issue_price` vs `issue_pricing`). Check column names in Catalog Explorer.
- Issue flags are strings (from `get_json_object`), so compare to `'true'`.

Widgets | Widget | Type | Dataset | Config | |---|---|---|---| | Restaurant | Filter: single/multi select on **field** restaurant_name | all datasets | "All" = whole chain | | Total Reviews | Counter | ds_restaurant_reviews | SUM(total_reviews) (same as none when 1 restaurant is selected) | | Avg Rating | Counter | ds_restaurant_reviews | AVG(avg_rating). Across "All" it's an average of averages; weighting by review count would be more precise | | City | Counter | ds_restaurant_reviews | city (None) | | Positive / Neutral / Negative | Counters | ds_restaurant_reviews | SUM(sentiment_count), colored green / yellow / red | | Review Sentiment Trend | Line | ds_review_trend | x = review_date (daily), y = positive / negative / neutral counts (green / red / yellow) | | Rating Distribution | Bar (horizontal) | ds_rating_distribution | y = rating_label, x = review_count, labels on | | Delivery / Food quality / Portion size / Pricing issues | Counters | ds_issue_categorization | SUM(count_issue) | | Recent Positive Reviews | Table | ds_recent_positive | review_text (green), review_timestamp | | Recent Negative Reviews | Table | ds_recent_negative | review_text (dark red), review_timestamp |

Why the trend matters: a sudden spike in negative reviews is the signal to step in and fix the root cause.

Publish. Both dashboards run inside Databricks on the lakehouse.

13. Recap, costs, troubleshooting, interview prep

13.1 What was built (60-second explanation)

1. **Two sources:** Event Hubs (real-time orders) + Azure SQL (master data, history, reviews).
2. **Two ingestion methods:** SDP streaming (Kafka endpoint) and Lakeflow Connect CDC (gateway + serverless pipeline).
3. **Medallion in Unity Catalog:** landing (staging volume) → bronze (raw) → silver (star schema facts + dims, expectations, ai_query sentiment) → gold (materialized views).
4. **Delta Lake** underneath everything.
5. **Jobs** orchestrate ingest → silver → gold.
6. **Databricks SQL + AI/BI dashboards** for consumption.

13.2 Cost checklist

- [] Event Hubs Standard; 1 event per batch; stop the producer when idle.
- [] Azure SQL **serverless**, LRS backups.
- [] **Cluster policy (1 worker)** on each Lakeflow Connect gateway, attached via CLI.
- [] **Stop gateways:** they're continuous.
- [] SQL warehouse 2X-Small with a short auto-stop.
- [] Dev cluster single-node, no Photon, auto-terminate.
- [] Cheapest pay-per-token model for ai_query. Streaming tables mean each review is scored once.
- [] Delete the resource group (and the workspace) at the end.

13.3 Troubleshooting (issues hit in the video, plus common ones)

Symptom	Fix
CSV import gives columns C1, C2...	Tick "first row is header"
Gateway cluster scales to 5 workers	Cluster policy + databricks pipelines update with policy_id, then restart the gateway

Symptom	Fix
SDP table created in <code>default</code> schema	Set the pipeline's default schema or use a fully qualified name
Kafka read returns no rows	Producer not running / wrong namespace / Send vs Listen policy mix-up / <code>startingOffsets</code>
<code>item_count</code> wrong	Use <code>size()</code> on arrays, not <code>count()</code>
"Cannot create streaming table append flow from batch query"	Wrap the source in <code>STREAM()</code> (SQL) / use <code>readStream</code> (Python)
Streaming table re-run shows 0 rows	Expected: the checkpoint already consumed the data
Alias not applied	<code>.alias()</code> goes on the aggregate expression, not inside <code>when</code>
UNRESOLVED_COLUMN in dashboard SQL	Check exact column names (e.g. <code>issue_pricing</code>)
MV recomputed fully	Check <code>planning_information</code> in the event log; consider partitioning/clustering

13.4 Interview questions with short answers

- Why Event Hubs Standard, not Basic?** Standard has the Kafka endpoint, so Spark's Kafka source can read Event Hubs with no brokers to manage.
- Why two SAS policies?** Least privilege: the producer can only Send, Databricks can only Listen. A leaked key has limited blast radius.
- What is `failOnDataLoss`?** When true, the query fails if expected offsets are gone (e.g. expired retention) instead of silently skipping data.
- `maxOffsetsPerTrigger`?** Caps records per micro-batch to control load and latency.
- Lakeflow Connect gateway vs pipeline?** The gateway continuously extracts snapshot + CDC into a UC staging volume on classic compute. The pipeline applies those changes to Delta tables on serverless.
- What must be enabled on SQL Server?** Change Tracking and/or CDC (plus the Databricks utility objects). Tables should have primary keys.
- How do you cap Lakeflow gateway cost?** A cluster policy attached through the CLI/API, and stop the gateway when not needed.
- Unity Catalog in one sentence?** Unified governance (access control, audit, lineage, discovery) for data and AI assets across workspaces, using `catalog.schema.object`.
- Metastore rules?** One per region; top-level metadata container; auto-created with the first workspace in a region.
- How would you design catalogs?** Business unit x environment (e.g. `sales_dev`, `sales_prod`), with a schema per project/use case.
- Declarative vs imperative pipelines?** Declarative defines tables and lets the engine handle DAG, order, checkpoints, retries and incremental processing. Imperative gives control but you write all the plumbing.
- Streaming table vs materialized view?** A streaming table is append/exactly-once over streaming input (ingestion, CDC). An MV is a query result kept current, possibly incrementally (aggregations, joins).
- Append flow vs AUTO CDC flow?** Append only adds rows. AUTO CDC applies inserts/updates/deletes with sequencing and maintains SCD1/SCD2.
- Is MV refresh always incremental?** No. The cost model picks incremental or full recompute. Check `planning_information` in the event log.
- Expectation actions?** Warn (keep), drop row, fail update. `expect_all_*` applies a dict of rules.

16. **How does `ai_query` work, and how do you control cost?** It calls a Model Serving endpoint per row from SQL. Control cost by using a cheap model, scoring only new rows (streaming table), keeping prompts short and structuring the output.
17. **How do you parse LLM JSON output?** `get_json_object(col, '$.key')`, or `from_json` with a `schema / responseFormat` for structured output.
18. **Why decimal for money?** Doubles have binary floating-point rounding errors. `decimal(10,2)` is exact.
19. **`explode` use case?** Turn an array of items per order into one row per item (`fact_order_items`).
20. **row_number vs rank vs dense_rank for "favorite"?** `row_number` always returns one row per partition. `rank/dense_rank` return ties.
21. **Why a left join from the dimension in gold tables?** To keep all entities (restaurants/customers) even with no facts, then `coalesce` NULLs to 0.
22. **A modeling issue in this project?** `fact_order_items` lacks `customer_id`, so favorite-item logic joins two large facts. Denormalize the key into the item fact.
23. **Parameter vs field filter in AI/BI?** A parameter changes the SQL (runs before the query). A field filter filters the returned results (fast interactivity).
24. **How do you unpivot in Spark SQL?** `LATERAL VIEW stack(n, 'label', col, ...)` or the `UNPIVOT` clause.
25. **Why is AOV computed from `fact_orders`, not averaged from daily AOVs?** An average of averages ignores differing order counts per day and gives the wrong result.
26. **How would you keep restaurant review history?** Add a `snapshot_date` and append (partitioned by it), or `MERGE` on `restaurant_id` with `SCD2`.
27. **How does the historical backfill interact with streaming?** A one-time `INSERT` into the streaming table. A full refresh would wipe it, so a `once=True` append flow is more robust.